

Parcial 2 — 2026

Sistemas Operativos / Sistemas Operativos y Redes

Tomás Rodeghiero

UNRC — Universidad Nacional de Río Cuarto

2026

Resumen

Resolución completa del Parcial 2 del año 2026 de la cátedra de Sistemas Operativos (Lic. en Ciencias de la Computación, UNRC). El parcial consta de cinco ejercicios que cubren: (i) verdadero/falso sobre gestión de heap, fragmentación, *best fit*, `mmap` y paginación; (ii) evolución de un *buddy system* con una secuencia de `malloc/free`; (iii) diseño de tabla de páginas para un proceso, traducción de direcciones e identificación de una página de guarda para detectar *stack overflow*; (iv) representación de un sistema de archivos FAT32 dado el árbol de un volumen pequeño con un archivo binario de 2 bloques y un directorio vacío; (v) verdadero/falso sobre privilegios de usuario, `sudo`, contraseñas y permisos UGO. Cada ejercicio se desarrolla con la estructura **Enunciado** → **Análisis** → **Desarrollo** → **Respuesta**.

Índice

1. Ejercicio 1 — V/F sobre gestión de memoria (1.5 pts)	1
2. Ejercicio 2 — Evolución de un <i>buddy system</i> (2 pts)	3
3. Ejercicio 3 — Paginación con páginas de 2 KB (3 pts)	5
4. Ejercicio 4 — Sistema de archivos FAT32 (2 pts)	7
5. Ejercicio 5 — V/F sobre usuarios y permisos (1.5 pts)	9
Resumen del parcial	10

1. Ejercicio 1 — V/F sobre gestión de memoria (1.5 pts)

Enunciado. Determinar la verdad o falsedad de las siguientes sentencias. Justificar las falsas en forma breve y con claridad.

1.a — Heap de bloques variables y fragmentación

“El gestor de un heap de bloques de tamaño variable tiene problemas de fragmentación.”

Respuesta

V. Un gestor de heap con bloques de tamaño variable (como el `malloc` clásico, `dldmalloc`, `ptmalloc`) sufre inherentemente **fragmentación externa**: con el correr de muchas asignaciones y liberaciones de tamaños distintos, los huecos libres se fragmentan en bloques pequeños no contiguos, y puede pasar que la suma total de memoria libre sea grande pero no haya un único hueco contiguo suficiente para satisfacer un pedido. También puede haber **fragmentación interna** si el gestor redondea cada pedido hacia arriba (a un múltiplo del tamaño mínimo).

1.b — Bitset para bloques de tamaño fijo

“Un gestor de bloques de tamaño fijo puede manejar los bloques libres/ocupados con un bitset.”

Respuesta

V. Cuando todos los bloques tienen el mismo tamaño, alcanza con **un bit por bloque** (0 = libre, 1 = ocupado). El *bitmap* (o *bitset*) es el método clásico de *slab allocators*, asignadores de *frames* de memoria física y administradores de *inodes*/bloques de disco.

1.c — Best fit y lista de libres ordenada

“La estrategia best fit en un `malloc(s)` asigna un bloque libre b con $\text{size}(b) \geq s$ y $\forall b': \text{free}(b') \wedge \text{size}(b') \geq \text{size}(b)$. Además conviene mantener la lista de bloques libres ordenadas de menor a mayor tamaño.”

Respuesta

V. *Best fit* busca, entre los bloques libres que satisfacen el pedido ($\text{size}(b) \geq s$), el de **menor tamaño** (es decir, el que mejor “calza”). La intuición es que esto deja los bloques grandes intactos para futuros pedidos grandes, reduciendo la probabilidad de fragmentar uno innecesariamente.

Mantener la lista de libres ordenada *de menor a mayor tamaño* es la implementación natural: se recorre la lista hasta encontrar el primer bloque $\geq s$ y ese es el *best fit*, en tiempo proporcional a la cantidad de bloques de tamaño menor que s . (En la práctica se usan estructuras más eficientes, como árboles balanceados ordenados por tamaño, para hacerlo en $O(\log n)$.)

1.d — mmap carga todo el archivo a memoria

“La llamada al sistema `mmap(file, ...)` carga todo el archivo en memoria y retorna la dirección lógica del comienzo del área de memoria asignada.”

Respuesta

F. `mmap` **no carga el archivo entero** en memoria física al momento de la llamada. Lo que hace es crear un *mapeo* en el espacio de direcciones virtuales del proceso: se agregan entradas en la tabla de páginas marcadas como *no presentes* ($P = 0$) que asocian rangos virtuales con bloques del archivo en disco. Las páginas se traen a memoria **bajo demanda** (*demand paging*): cuando el proceso accede a una dirección dentro del mapeo, el hardware genera un *page fault*; el kernel atiende la falla leyendo el bloque correspondiente del archivo y poblando la página.

Sí es cierto que `mmap` devuelve la dirección lógica del comienzo del área mapeada (eso está bien). El problema de la afirmación es la palabra “carga todo el archivo”: la carga es *perezosa*, no inmediata.

1.e — Ventaja de la paginación

“La paginación tiene la ventaja que los bloques de un proceso en memoria (código, datos globales, *stack*, etc) no requieren estar físicamente contiguos.”

Respuesta

V. Ésta es la ventaja característica de la paginación. El espacio de direcciones virtual del proceso es *lógicamente contiguo*, pero la MMU traduce cada página virtual a una página física independiente: las páginas físicas de un mismo proceso pueden estar dispersas en cualquier parte de la RAM. Esto elimina la fragmentación externa que sufren las particiones variables y permite asignar memoria a cualquier proceso siempre que haya *frames* libres, sin importar su distribución espacial.

2. Ejercicio 2 — Evolución de un *buddy system* (2 pts)

Enunciado. Mostrar la evolución de los bloques ocupados y libres de un *buddy system* con bloques de tamaño entre 2^6 y 2^9 . El tamaño del heap es de 2 KB. Secuencia de operaciones:

- I. `ptr1 = malloc(500)`
- II. `ptr2 = malloc(8)`
- III. `ptr3 = malloc(120)`
- IV. `free(ptr2)`
- V. `ptr4 = alloc(600)`

Interpretación del enunciado

El heap mide $2\text{ KB} = 2^{11}$ bytes y los tamaños de bloque mencionados son $2^6 = 64$ y $2^9 = 512$. Tomamos esto como una indicación de la *granularidad típica* de las asignaciones: el heap se inicializa como un único bloque grande de $2^{11} = 2048$ bytes y el *buddy system* puede subdividirlo recursivamente hasta alcanzar el mínimo de 64 bytes, o consolidar bloques contiguos hasta el máximo de 2048.

Idea intuitiva

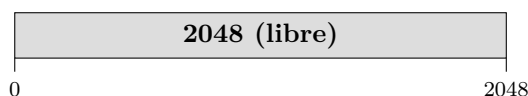
Bajo esta interpretación, el último pedido `alloc(600)` se satisface con un bloque de $1024 = 2^{10}$ bytes y el ejercicio muestra toda la mecánica del buddy. Si en cambio se interpretara estrictamente que el bloque máximo es $2^9 = 512$, el último pedido fallaría por no haber ningún bloque de 1024 disponible; la versión adoptada acá es la pedagógicamente más rica.

Algoritmo del *buddy system*

- Para un pedido de tamaño s , se busca el menor bloque libre de tamaño 2^k con $2^k \geq s$.
- Si el bloque encontrado es *demasiado grande*, se divide a la mitad recursivamente hasta llegar al tamaño justo.
- Al liberar un bloque, se verifica si su *buddy* (el bloque vecino del mismo tamaño con el bit complementario de offset) está libre; si lo está, se fusionan en un bloque del doble del tamaño. La fusión se repite mientras sea posible.

Estado inicial

Heap libre de 2048 bytes en el offset 0.



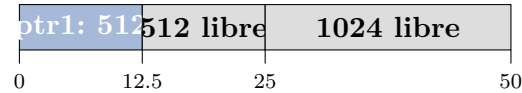
Paso I: `ptr1 = malloc(500)`

500 bytes redondeado hacia la siguiente potencia de 2 que sea ≥ 500 : es $2^9 = 512$.

Necesitamos un bloque libre de 512. El único bloque libre es de 2048. Lo dividimos sucesivamente:

```
2048 -> 1024 + 1024 (tomo el primer 1024)
1024 -> 512 + 512 (tomo el primer 512)
```

Asigno el bloque de 512 en offset 0 a `ptr1`.



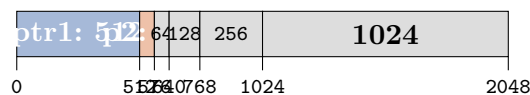
Paso II: `ptr2 = malloc(8)`

8 bytes redondeado al mínimo permitido $2^6 = 64$.

El bloque libre más chico disponible es 512 (en offset 512). Subdividimos hasta llegar a 64:

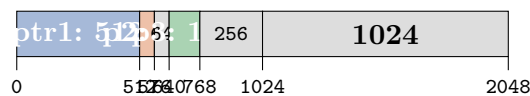
```
512 -> 256 + 256 (tomo el primer 256, offset 512)
256 -> 128 + 128 (tomo el primer 128, offset 512)
128 -> 64 + 64 (tomo el primer 64, offset 512)
```

`ptr2` apunta al bloque de 64 en offset 512.



Paso III: `ptr3 = malloc(120)`

120 se redondea a $2^7 = 128$. Hay un bloque libre de 128 en offset 640 que coincide exactamente. Lo asigno a `ptr3`, sin necesidad de subdividir.

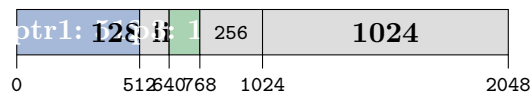


Paso IV: `free(ptr2)`

Libero el bloque de 64 en offset 512. El buddy de un bloque de tamaño 64 en offset 512 está en offset $512 \oplus 64 = 576$. El bloque en 576 (de 64 bytes) está libre → **fusiono**:

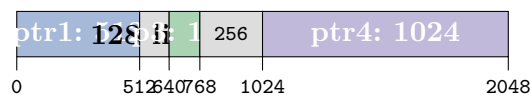
`libre 64@512 + libre 64@576 → libre 128@512`

Intento volver a fusionar el nuevo bloque de 128 en offset 512. Su buddy está en offset $512 \oplus 128 = 640$. Pero en 640 está `ptr3` (ocupado). **No se puede fusionar más.**



Paso V: `ptr4 = alloc(600)`

600 se redondea a $2^{10} = 1024$. Hay un bloque libre de 1024 exactamente en offset 1024. Lo asigno a `ptr4`.



Estado final — resumen

Offset	Tamaño	Estado	Apunta a / nota
0	512	ocupado	ptr1 (pedido original: 500 B; frag. interna 12 B)
512	128	libre	— (proviene de la fusión del paso IV)
640	128	ocupado	ptr3 (pedido: 120 B; frag. interna 8 B)
768	256	libre	—
1024	1024	ocupado	ptr4 (pedido: 600 B; frag. interna 424 B)

Respuesta

Punteros activos al final:

- ptr1 → bloque de 512 bytes en offset 0.
- ptr2 → liberado (no apunta a nada válido).
- ptr3 → bloque de 128 bytes en offset 640.
- ptr4 → bloque de 1024 bytes en offset 1024.

Bloques libres: 128 B en offset 512 y 256 B en offset 768. Memoria libre total: 384 bytes (pero *ningún bloque contiguo* mayor a 256).

3. Ejercicio 3 — Paginación con páginas de 2 KB (3 pts)

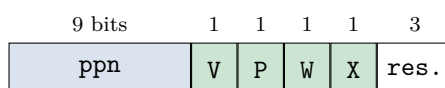
Enunciado. CPU de 32 bits con espacio de direcciones de 1 MB = 2^{20} y páginas de 2 KB.

Análisis de los parámetros básicos

- Tamaño de página: 2 KB = 2^{11} bytes.
- Bits para el **offset** dentro de la página: 11.
- Bits para el **vpn** (*virtual page number*): $20 - 11 = 9$.
- Cantidad de páginas en el espacio total: $2^{20}/2^{11} = 2^9 = 512$.

3.I — Formato de la tabla de páginas

- **Cantidad de entradas:** $2^9 = 512$.
- Cada entrada (**pte**) almacena:
 - **ppn** (*physical page number*): si el espacio físico también es de 1 MB, hacen falta 9 bits.
 - **flags:** *V* (válida), *P* (presente en RAM), *W* (escribible), *X* (ejecutable). Son 4 bits.
- Total por entrada: $9 + 4 = 13$ bits útiles; redondeado a 16 **bits (2 bytes)** por motivos de alineación.
- **Tamaño de la tabla:** $512 \times 2 \text{ B} = 1024 \text{ B} = 1 \text{ KB}$. Cabe holgadamente en una página.



3.II — Interpretación de una dirección virtual

Las direcciones virtuales tienen 20 bits útiles. La MMU las parte en dos campos:



- Los **9 bits altos** forman el **vpn**: indexan una de las 512 entradas de la tabla.
- Los **11 bits bajos** son el **offset** dentro de la página de 2 KB.

3.III — Tabla de páginas de un proceso concreto

Datos del proceso.

- Espacio de direcciones: $[0, 512 \text{ KB})$. Esto son $512 \text{ KB} \div 2 \text{ KB} = 256$ páginas (vpn 0 a 255).
- 2 páginas de código $\rightarrow$ vpn 0 y 1.
- 1 página de datos globales $\rightarrow$ vpn 2.
- 2 páginas de stack *al final del espacio* $\rightarrow$ vpn 254 y 255.

Tabla. Los ppn concretos los asigna el SO según los frames físicos libres; usamos valores arbitrarios como ejemplo.

vpn	ppn	V	P	W	X	U	Área
0	0x010	1	1	0	1	1	código (página 1)
1	0x011	1	1	0	1	1	código (página 2)
2	0x020	1	1	1	0	1	datos globales
3...253	—	0	0	0	0	0	inválido (gap)
254	0x030	1	1	1	0	1	stack (página 1)
255	0x031	1	1	1	0	1	stack (página 2)

Justificación de las flags:

- **Código:** $V = P = 1$, $W = 0$ (no se modifica), $X = 1$ (ejecutable).
- **Datos / stack:** $V = P = 1$, $W = 1$ (escribible), $X = 0$ (no se ejecuta como código; protección contra explotación).
- **Gap (3–253):** $V = 0$. Cualquier acceso a una de esas direcciones genera un *page fault*.

3.IV — ¿A qué página física se mapea 0x2412?

Procedimiento. Decodificar 0x2412 usando los 9 bits altos para vpn y 11 bits bajos para offset.

$$0x2412 = 0000\ 0010\ 0100\ 0001\ 0010_2 \text{ (20 bits)}$$

$$\text{vpn (9 bits altos)} = 0\ 0000\ 0100_2 = 4$$

$$\text{offset (11 bits bajos)} = 100\ 0001\ 0010_2 = 0x412 = 1042$$

Alternativa numérica:

$$0x2412 \div 0x800 = 9234 \div 2048 = 4 \text{ (cociente)}$$

$$0x2412 \bmod 0x800 = 9234 - 4 \times 2048 = 1042 = 0x412$$

Consulta a la tabla. La página $\text{vpn} = 4$ está en el rango $[3, 253]$ del *gap*, donde todas las entradas tienen $V = 0$.

Respuesta

(3.IV) La dirección 0x2412 **no se mapea a ninguna página física**: cae en $\text{vpn} = 4$, que pertenece al **gap inválido** entre los datos globales y el stack. El acceso generaría un *page fault* ($V=0$) y el kernel enviaría SIGSEGV al proceso. No corresponde a código, datos ni stack.

Nota didáctica: el ejercicio testea que el estudiante *verifique* si la página es válida antes de asumir el área. Si la página fuese de 4 KB (no 2 KB), la dirección 0x2412 caería en $\text{vpn} = 2$, que es la página de datos; pero con 2 KB cae en el gap.

3.V — Página de guarda para detectar stack overflow

Idea. Si el SO permite que el stack crezca dinámicamente hacia direcciones más bajas (asignando bajo demanda las páginas 253, 252, ...), conviene reservar una página **inmediatamente**

después de los datos globales como *guarda*: nunca se mapea ($V = 0$), y cualquier intento de tocarla provoca un page fault que el kernel detecta como *stack overflow*.

Elección concreta. La página de guarda más natural es $\text{vpn} = 3$, justo después de los datos ($\text{vpn} = 2$):

- Marcar $\text{vpn} = 3$ con $V = 0$ **permanentemente**.
- Cuando el stack baja a $\text{vpn} = 4$, todavía hay espacio.
- Cuando intenta acceder a $\text{vpn} = 3$, el hardware levanta un fault, el kernel reconoce que la dirección está “por debajo” del límite válido del stack y *aborta* el proceso con *stack overflow* **antes** de pisar los datos.

Espacio de direcciones lógicas que caen en la guarda. $\text{vpn} = 3$ con páginas de 2 KB cubre el rango:

$$[3 \times 2048, 4 \times 2048) = [6144, 8192) = [0x1800, 0x2000)$$

Respuesta

(3.V) La entrada $\text{vpn} = 3$ debe quedar sin mapear ($V = 0$) y servir como guarda. El espacio de direcciones lógicas inválidas asociado es:

$$[0x1800, 0x2000)$$

es decir, los 2 KB que ocuparía esa página entre el área de datos y la zona donde podría crecer el stack.

4. Ejercicio 4 — Sistema de archivos FAT32 (2 pts)

Enunciado. Completar el diagrama del sistema de archivos FAT32 con bloques de datos de 512 bytes, correspondiente al árbol:

```
\
|-- shell.com (programa, 2 bloques de datos)
+-- docs (directorio vacío)
```

Cada entrada de directorio ocupa 32 bytes.

Análisis previo: cuántos bloques ocupa cada cosa

Tamaño de los datos de cada entrada del árbol.

- $\backslash$ (root directory): contiene 4 entradas ($\cdot$, $\dots$, *shell.com*, *docs*). $4 \times 32 = 128$ bytes. Cabe en 1 **bloque** de 512.
- *shell.com*: 750 bytes según la tabla de root. $\lceil 750/512 \rceil = 2$ bloques.
- *docs*: directorio vacío significa que contiene sólo $\cdot$ y $\dots$: $2 \times 32 = 64$ bytes. Cabe en 1 **bloque**.

Total ocupado: $1 + 2 + 1 = 4$ bloques (de 5 disponibles).

Asignación propuesta de clusters

# de cluster	Contenido
1	directorio raíz ($\backslash$)
2	<i>shell.com</i> , bloque 1 (bytes 0–511)
3	directorio <i>docs</i>
4	<i>shell.com</i> , bloque 2 (bytes 512–749 y <i>padding</i>)
5	libre

Tabla FAT completada

Cada entrada de la FAT dice qué cluster sigue al actual, o si es el final de la cadena (EOC) o si está libre (0).

FAT				
EOC	4	EOC	EOC	0
1	2	3	4	5

Justificación entrada por entrada:

- FAT[1] = EOC: el directorio raíz ocupa exactamente un bloque, así que la cadena termina en cluster 1.
- FAT[2] = 4: `shell.com` comienza en cluster 2 y *encadena* al cluster 4 para su segundo bloque (notar que 3 está ocupado por `docs`).
- FAT[3] = EOC: `docs` ocupa un solo bloque.
- FAT[4] = EOC: cluster 4 es el último bloque de `shell.com`.
- FAT[5] = 0: cluster libre.

Contenido de los bloques de datos

Cluster 1 (directorio raíz \):

nombre	tipo / tamaño	1.º cluster
.	dir, size = 128	1
..	dir, size = 128	1
shell.com	file, size = 750	2
docs	dir, size = 64	3

Notar que `..` de la raíz apunta a sí misma (cluster 1), porque la raíz no tiene padre.

Cluster 2: primeros 512 bytes del programa `shell.com`.

Cluster 3 (directorio `docs`):

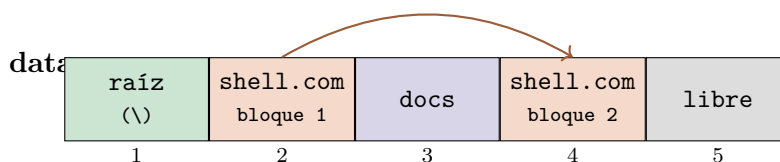
nombre	tipo / tamaño	1.º cluster
.	dir, size = 64	3
..	dir, size = 128	1

`..` de `docs` apunta a la raíz (cluster 1); `.` apunta al propio cluster del directorio.

Cluster 4: los 238 bytes finales de `shell.com` (bytes 512–749) más *padding* hasta completar los 512 bytes del cluster.

Cluster 5: libre, sin contenido válido.

Diagrama final del disco



La flecha arriba muestra el encadenamiento de `shell.com`: cluster 2 → cluster 4 (saltando cluster 3 que está ocupado por `docs`). Este “salto” es justamente la utilidad de la FAT: la tabla permite fragmentar archivos a lo largo del disco sin que el cliente se entere.

Respuesta

La tabla FAT queda [**EOC, 4, EOC, EOC, 0**] para los clusters 1 a 5. Los bloques de datos contienen, en orden: directorio raíz, primer bloque de `shell.com`, directorio `docs`, segundo bloque de `shell.com`, libre.

Idea intuitiva

La elección de poner `docs` en el cluster 3 (en lugar de poner `shell.com` consecutivo en clusters 2–3) ilustra cómo la fragmentación natural del filesystem se gestiona con la FAT. En un disco real con miles de archivos creándose y borrándose, esta estructura permite ubicar bloques en cualquier lado y reconstruir la cadena en tiempo $O(\text{longitud})$ con la tabla.

5. Ejercicio 5 — V/F sobre usuarios y permisos (1.5 pts)

Enunciado. Determinar la verdad o falsedad de las siguientes sentencias. Justificar las falsas.

5.a — “Un proceso se ejecuta con los privilegios del usuario real”

Respuesta

F. Un proceso se ejecuta con los privilegios del **usuario efectivo** (`euid`), no del real (`ruid`). En un proceso “normal” los dos coinciden, pero el modelo UNIX deliberadamente los separa para soportar programas con **bit SUID**: cuando se ejecuta un binario con `u+s`, el kernel pone como `euid` del proceso el dueño del archivo ejecutable, no el del usuario que lo lanzó. Ejemplo clásico: `/usr/bin/passwd` pertenece a `root` con el bit SUID; cuando un usuario común lo ejecuta, su `ruid` sigue siendo el suyo pero su `euid` pasa a ser `root`, lo que le permite escribir en `/etc/shadow`. Los chequeos de permisos del kernel se hacen contra `euid`, no contra `ruid`.

5.b — “`sudo` solicita la contraseña del usuario ejecutor”

“El comando `sudo` permite ejecutar un programa con los privilegios de otro usuario y solicita la contraseña del usuario ejecutor.”

Respuesta

V. Las dos partes son correctas:

- `sudo` permite ejecutar un comando como otro usuario (por default `root`; con `-u` se elige otro target).
- Por defecto, `sudo` pide la **contraseña del usuario que lo invoca** (*the user typing `sudo`*), no la del usuario objetivo. La verificación es “demostrame que sos vos quien está pidiendo el privilegio”; el archivo `/etc/sudoers` decide a quién está autorizado a elevarse.

Nota: existe la opción `Defaults targetpw` que invierte el comportamiento (pide la del target), pero no es el default.

5.c — “Es posible recuperar la contraseña. . .”

“Es posible recuperar la contraseña de un usuario ya que el sistema la conoce.”

Respuesta

F. El sistema **no conoce la contraseña en texto plano**. Lo que guarda en `/etc/shadow` es un **hash** criptográfico (típicamente `bcrypt`, `argon2`, `sha512crypt`) con *salt*. La función hash es de un solo sentido: dado el hash es computacionalmente inviable recuperar la contraseña original.

Cuando el usuario hace login, el sistema toma la contraseña que ingresa, le aplica la misma función hash con el mismo salt, y compara con el hash guardado. Si coinciden, autentica. Por eso, si un usuario olvida su contraseña, el administrador no puede “ver cuál era”; solo puede **asignarle una nueva** (`passwd <usuario>`). Esto es una propiedad de seguridad deliberada: aún si un atacante roba `/etc/shadow`, no obtiene contraseñas en claro.

5.d — Permisos `rw-rw-r--`

“Un archivo con permisos `rw-rw-r--` puede ser escrito por el dueño y cualquier usuario del grupo asignado al archivo pero no por el resto de usuarios.”

Respuesta

V. Decodifiquemos el modo `rw-rw-r--`:

	Categoría	Permisos	Octal	Significado
	Owner (u)	<code>rw-</code>	6	leer y escribir
	Group (g)	<code>rw-</code>	6	leer y escribir
	Other (o)	<code>r--</code>	4	solo leer

Total: 664 en octal. Por tanto:

- El **dueño** puede escribir (*w* en u).
- Los miembros del **grupo asignado** al archivo pueden escribir (*w* en g).
- Los **demás usuarios** (no dueño ni del grupo) solo pueden leer.

La afirmación coincide exactamente. **V.**

Resumen del parcial

#	Tema	Respuesta sintética
1	V/F memoria	a:V, b:V, c:V, d:F (<code>mmap</code> es lazy), e:V
2	Buddy system	<code>ptr1@0</code> (512), <code>ptr3@640</code> (128), <code>ptr4@1024</code> (1024); libres: <code>128@512</code> y <code>256@768</code>
3	Paginación	512 ptes, vpn 9 bits + offset 11 bits; tabla con código 0–1, datos 2, stack 254–255, gap 3–253; <code>0x2412</code> cae en gap (page fault); guarda en vpn 3 con rango <code>[0x1800, 0x2000)</code>
4	FAT32	FAT = [EOC, 4, EOC, EOC, 0]; raíz @1, shell.com @2 → 4, docs @3
5	V/F usuarios	a:F (<code>uuid</code> , no <code>ruid</code>); b:V; c:F (hash); d:V